

YMPrint Slides

Desktop publishing with YAML, now in 16:9.

Connor Ferster · Structural Python

A deck is just a YAML file

- ▶ Every top-level heading is **one slide**
- ▶ Frames place content side by side
- ▶ Backgrounds are ordinary PDFs, drawn in anything
- ▶ Renders to a crisp **vector** PDF

slides.yml — this very slide

A deck is just a YAML file:

- _slide:
 - layout: code-right
 - template: content
- _textstyle: small
- _ul:
 - Every top-level heading is ****one slide****
 - Frames place content side by side
 - Backgrounds are ordinary PDFs, ...
 - Renders to a crisp ****vector**** PDF
- _frame: code
- _code:
 - caption: slides.yml — this very slide
 - source: ...

01 Layouts & frames

Put anything anywhere, as long as it is a rectangle.



Layouts are named rectangles

Define a layout once under `_doc`, then pick it per slide with `_slide`.

Each frame is `[x, y, width, height]`, measured from the top-left of the content box.

Send content to a frame with `_frame`, and it sizes itself to fit.

`_doc → layouts`

`layouts:`

`two-column:`

`title: {box: [0, 0, 1, 150],
valign: bottom}`

`left: [0, 190, 0.47, 690]`

`right: [0.53, 190, 0.47, 690]`

Images drop into any frame

Our logo, faithfully preserved at full **year 2000** fidelity.

An `_img` with `width_ratio: 1` fills its frame. A frame with `valign: middle` centres it.

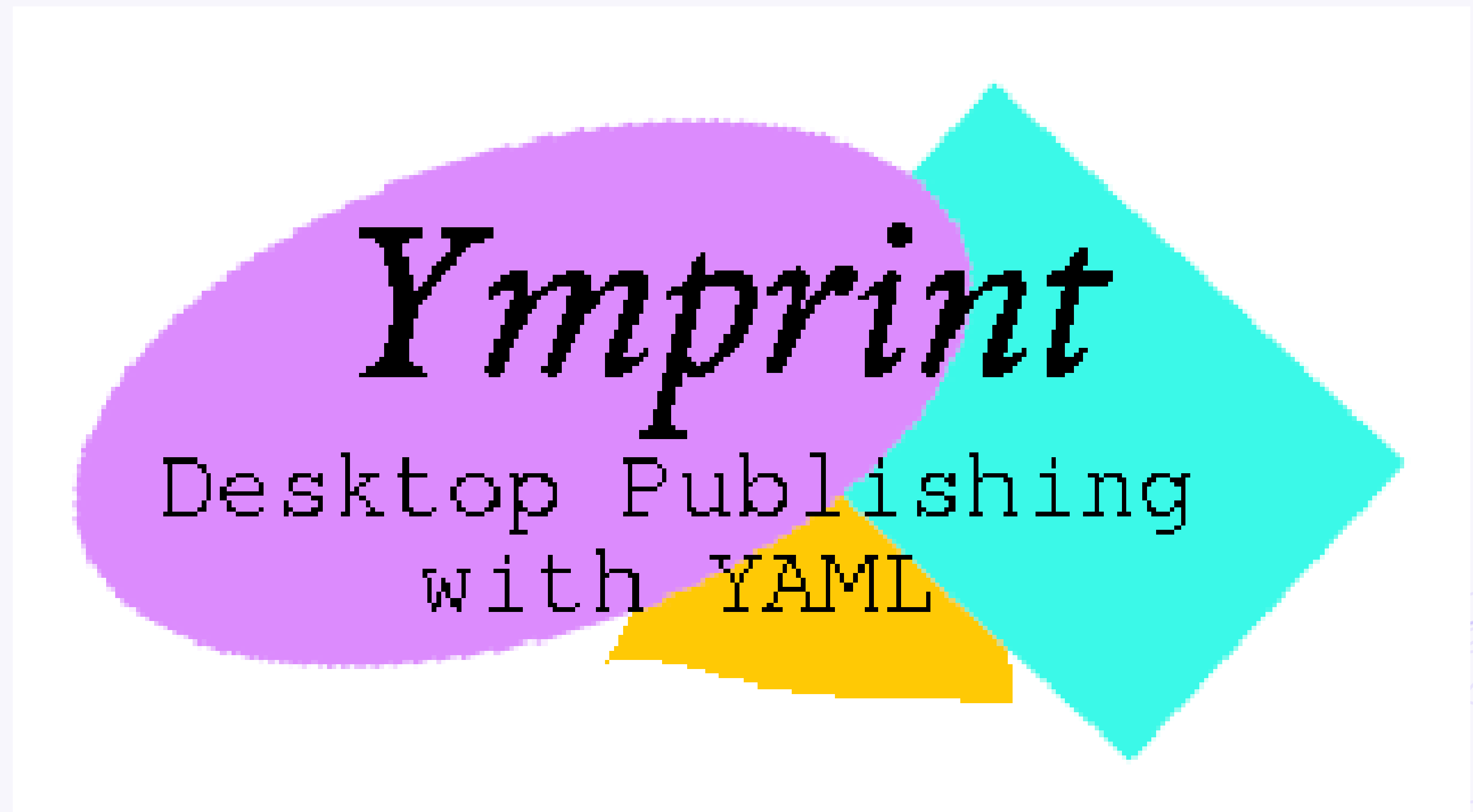


Figure 1: Desktop publishing, circa 2000

02 Code that runs

Python executes while the PDF renders. The results land on the slide.



Run it, then show it: `_py`

Executed during ym convert

```
1 w = 12.0      # kN/m, uniform load
2 L = 8.0       # m, span
3 E = 200e6     # kN/m2, steel
4 I = 3.0e-4    # m4
5 M_max = w * L**2 / 8
6 V_max = w * L / 2
7 delta = 5*w*L**4 / (384*E*I) * 1000
8 limit = L * 1000 / 360
9 ok = delta < limit
```

The variables flow straight into a table with `{{ beam.M_max }}`:

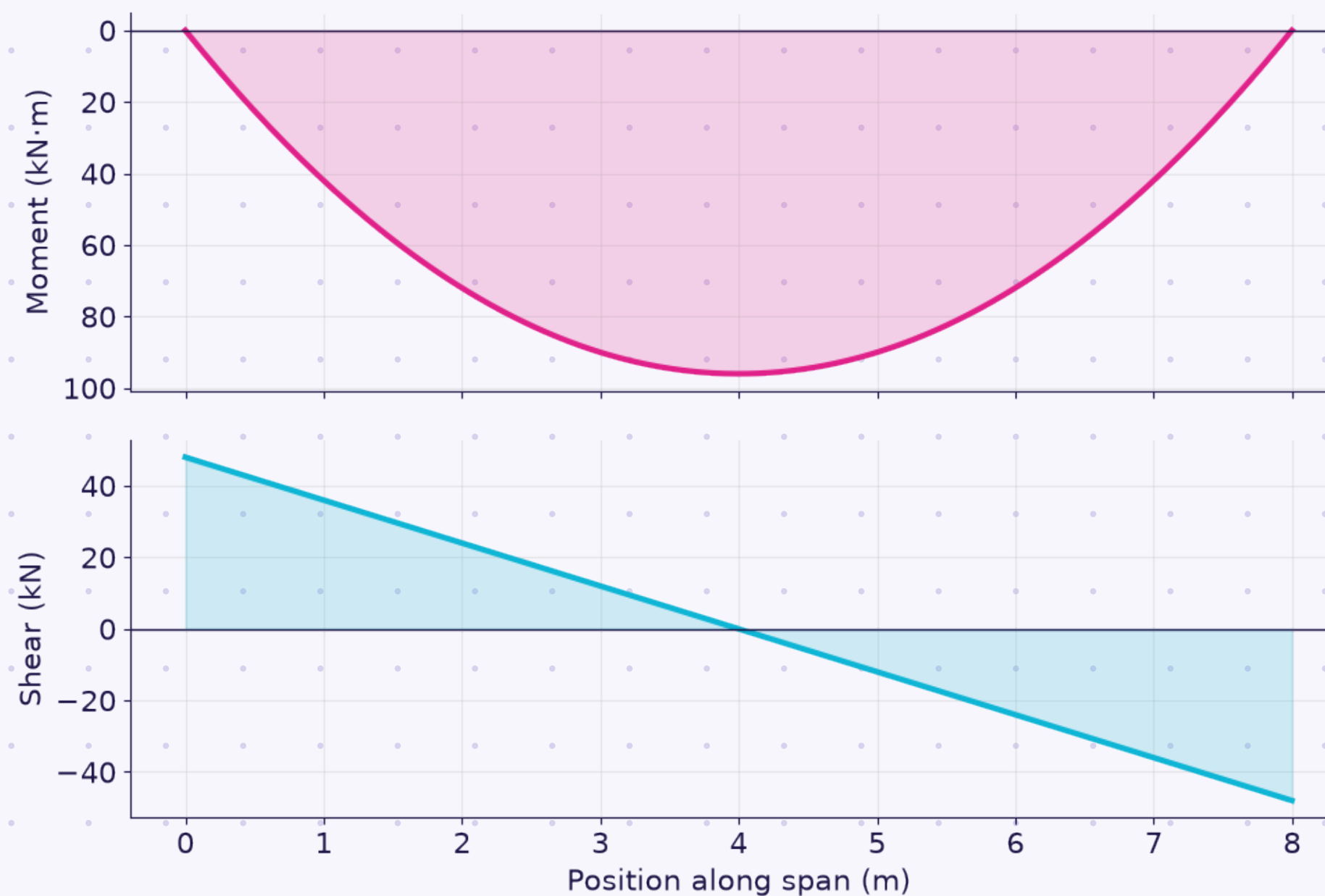
Quantity	Value
Max. moment	96.0 kN·m
Max. shear	48.0 kN
Deflection	10.7 mm
Limit (L/360)	22.2 mm
Check	PASS

Plots from live objects: `_matplotlib`

A hidden `_py` block builds a matplotlib figure from the beam on the last slide.

`$bmd_fig` hands the live object to the block:

```
_matplotlib:  
  fig: $bmd_fig  
  scale_ratio: 1
```



03 Words, lists & tables

Everything a document can do, a slide can do.



Inline markup & variables

Write **bold**, *italic* and `inline code` in plain YAML strings.

Variables interpolate anywhere: this deck honours the year 2000.

ReportLab markup passes through, for **colour** and **size**.

`_p` picks a named style:
this is “callout”.

The source of the text on the left

- Write **bold**, *italic* and `inline code` in plain YAML strings.
- "Variables interpolate anywhere: this deck honours the year {{year}}."
- "ReportLab markup passes through, for `<font color='#e0218a'>colour</font>` and `<font size='44'>size</font>`."
- `_p`:
 - style: callout
 - content: "`_p` picks a named style: ..."

Callouts scale with your text

✓ TIP

Admonitions, quotes and captions size themselves from the body text, so they read on a 1080p slide.

■ WARNING

This deck has a 38 pt body, so these callouts are drawn 3.8 times their size in a 10 pt report.

✍ NOTE

Five kinds are available — `_info`, `_warning`, `_danger`, `_tip` and `_note`.

“What you do makes a difference, and you have to decide what kind of difference you want to make.”

— Jane Goodall

Lists, ordered and not

- ▶ Gathering information

- ▷ Site visits
- ▷ Drawings and specs

- ▶ Simulation

- ▷ Hand calcs
- ▷ Finite element models

- ▶ Reporting

1. Write YAML

2. Run `ym convert`

3. Present

1. Full screen

2. Any PDF viewer

Tables are lists of mappings

Every mapping is a row, and its keys are the column headers.

Header and row colours come from `_tablestyle`.

Year	Technology	Role in this slide
1984	PostScript	The page description idea
1993	PDF 1.0	The file you are looking at
2000	ReportLab & Python 2.0	The renderer and its language
2001	YAML	The source format
2026	YMPrint slides	The glue

Thank you

```
$ ym convert slides.yml
```

The technology of the year 2000... today!

